

MarketIQ

Technical Architecture Document

Document Type	Version	Status	Date
Technical Architecture	v1.0	Active	April 2025
Cloud	AI Engine	Architecture	Auth
GCP	Claude API	Microservices + Serverless	Firebase Auth

Audience: Engineering leads, senior developers, DevOps engineers, solution architects.

Scope: Full platform — frontend, backend microservices, data layer, infrastructure, security, CI/CD.

Purpose: Single source of truth for all architectural decisions made for MarketIQ v1.

Table of Contents

1. Architectural Overview

MarketIQ is built as a cloud-native, serverless-first microservices platform on Google Cloud Platform. Every architectural decision is driven by three constraints: multi-tenant data isolation, white-label deployability, and cost efficiency at scale.

The platform is decomposed into independently deployable services, each owning its own data schema, GraphQL subgraph, and Cloud Run deployment. Services communicate synchronously via Apollo Federation GraphQL (user-facing requests) and asynchronously via Cloud Pub/Sub (background processing).

1.1 Architectural Principles

Principle	Rationale & Implementation
Serverless-first	All services deploy to Cloud Run with scale-to-zero. No idle compute costs. Workers run as Cloud Functions — pay only per invocation.
Tenant isolation at the data layer	Row-level security injected at query time. Snowflake workspace-per-tenant. Cross-tenant data access is architecturally impossible.
Config-over-customisation	Zero custom code per tenant. All tenant behaviour driven by a config object — branding, locale, SSO, permissions.
Event-driven background processing	User requests never wait for heavy work. Connectors, exports, notifications, and provisioning are async via Pub/Sub.
GraphQL Federation	Each microservice owns a subgraph. Apollo Router stitches them into a single unified API. No direct service-to-service HTTP calls.
Infrastructure as code	All GCP resources defined in Terraform. Tenant provisioning is a Terraform workflow, not a manual process.
Separation of analytical and operational data	Snowflake handles all marketing analytics (OLAP). Cloud SQL PostgreSQL handles application state (OLTP). They never cross.

1.2 High-Level Architecture

The platform is structured in six discrete layers, each with clear ownership and interfaces:

Frontend Layer	React + Vite SPA · Apollo Client · shadcn/ui · react-grid-layout dashboard canvas
API Gateway Layer	Apollo Federation Router · Single GraphQL endpoint · Auth token validation · Rate limiting · Tenant context injection
Service Layer	10 Node.js microservices · Each owns a GraphQL subgraph · Cloud Run (scale to zero) · Prisma ORM per service

Worker Layer	5 Cloud Functions · Event-driven via Pub/Sub · Sync triggers, PDF export, notifications, provisioning, baseline computation
Data Layer	Snowflake (analytics) · Cloud SQL PostgreSQL (operational) · Memorystore Redis (cache/queue) · GCS (data lake)
Infrastructure Layer	GCP · Cloud CDN · Firebase Auth · Secret Manager · Cloud Armor · Artifact Registry · Terraform IaC

1.3 Request Flow

Two distinct paths handle all platform traffic:

Synchronous Path — User-Facing Requests

```

Browser / Mobile App
  ↓ HTTPS · TLS 1.3
Cloud CDN + Cloud Load Balancer (SSL termination · WAF via Cloud Armor · DDoS protection)
  ↓
Apollo Federation Router (Cloud Run · Auth token validation · Tenant context injection · Rate limiting)
  ↓ GraphQL subgraph routing
Microservice (e.g. Dashboard Service · Query Service · AI Service)
  ↓ Prisma ORM or Snowflake SDK
Cloud SQL PostgreSQL OR Snowflake Gold Layer (tenant_id + scope injected on every query)
  ↓
Response → Router → CDN → Client
  
```

Asynchronous Path — Background Processing

```

Microservice publishes event (e.g. 'connector.authenticated', 'tenant.created')
  ↓
Cloud Pub/Sub topic (durable · at-least-once delivery · dead-letter queue)
  ↓
Cloud Function worker (e.g. Provisioning Worker · Sync Trigger · Notification Worker)
  ↓ side effects only
Airbyte pipeline / Snowflake write / Email via Resend / PDF via Puppeteer
  ↓
Worker publishes completion event back to Pub/Sub if downstream work needed
  
```

2. Service Architecture

Each microservice is an independent Node.js + TypeScript application with its own: Apollo Server subgraph, Prisma schema, Cloud Run deployment, Pub/Sub subscriptions, and Dockerfile. Services never query each other's databases — all cross-service communication goes via GraphQL or Pub/Sub.

2.1 Service Registry

Service	Type	Owns	Cloud Run Config
Apollo Federation Gateway	Routing	No DB — routing only	Min replicas: 1 · Always-on
Auth Service	API	Firebase Auth wrapper · Session tokens	Min: 0 · Scale-to-zero
Tenant Service	API	tenants · sub_tenants · tenant_configs	Min: 0 · Scale-to-zero
User Service	API	users · roles · scopes · invitations	Min: 0 · Scale-to-zero
Dashboard Service	API	dashboards · templates · schedules	Min: 0 · Scale-to-zero
Widget Service	API	widgets · widget_configs · widget_types	Min: 0 · Scale-to-zero
Query Service	API	No DB · Snowflake queries + Redis cache	Min: 0 · Scale-to-zero
AI Service	API	No DB · Claude API calls · spec cache	Min: 0 · Scale-to-zero
Connector Service	API	connectors · sync_jobs · connector_logs	Min: 0 · Scale-to-zero
Recommendation Service	API	No DB · Reads Gold layer baselines	Min: 0 · Scale-to-zero

2.2 Worker Registry

Worker	Trigger	Responsibility
Provisioning Worker	Pub/Sub: tenant.signup.completed	Run full tenant setup pipeline: Snowflake workspace, config seed, RBAC seed, subdomain, SSO config
Sync Trigger	Cloud Scheduler (per-tenant config)	Fire Airbyte connector pulls for all active tenants. Publish sync.started events.

Export Worker	Pub/Sub: export.requested	Generate PDF snapshots of dashboards using Puppeteer. Upload to GCS. Email link to recipients.
Notification Worker	Pub/Sub: notification.send	Route transactional emails via Resend. Handle anomaly alerts, onboarding nudges, sync completion.
Baseline Computer	Cloud Scheduler (nightly 02:00 UTC)	Recompute GOLD_TREND_BASELINE rolling averages for all active tenants in Snowflake.

2.3 Per-Service Structure

Every service follows an identical directory structure. This is enforced by a scaffold generator in the monorepo:

```
apps/services/[service-name]/
├── src/
│   ├── schema/
│   │   ├── types.ts           ← Pothos type definitions
│   │   ├── queries.ts         ← GraphQL queries
│   │   ├── mutations.ts       ← GraphQL mutations
│   │   └── index.ts           ← Schema builder export
│   ├── resolvers/
│   │   ├── query.resolvers.ts
│   │   └── mutation.resolvers.ts
│   ├── services/              ← Business logic (no GraphQL awareness)
│   │   └── [entity].service.ts
│   ├── db/
│   │   ├── schema.prisma      ← Service-owned Prisma schema
│   │   └── client.ts          ← Singleton Prisma client
│   ├── pubsub/
│   │   ├── publishers.ts      ← Events this service publishes
│   │   └── subscribers.ts     ← Events this service consumes
│   ├── middleware/
│   │   ├── auth.ts            ← Token validation middleware
│   │   └── tenant.ts           ← Tenant context injection
│   └── index.ts                ← Apollo Server entriypoint
├── Dockerfile
├── package.json
└── tsconfig.json
```

2.4 Service Communication Rules

- Rule 1: Services NEVER make direct HTTP calls to other services.
- Rule 2: Synchronous cross-service data needs → GraphQL query via Apollo Federation (entity references).
- Rule 3: Async side-effect triggers → publish to Pub/Sub topic.
- Rule 4: Services NEVER read from another service's database schema.
- Rule 5: The Query Service is the ONLY service that connects to Snowflake.
- Rule 6: The AI Service is the ONLY service that calls the Claude API.

3. GraphQL API Architecture

MarketIQ uses Apollo Federation v2. Each microservice publishes a GraphQL subgraph. The Apollo Router (gateway) composes all subgraphs into a unified supergraph schema exposed to the frontend. Clients query one endpoint and receive data from multiple services in a single round-trip.

3.1 Federation Architecture

```
Client → POST https://api.marketiq.app/graphql
      ↓
Apollo Router (Federation Gateway)
      ↓ query planning
┌──────────┬──────────┬──────────┬──────────┐
│ Auth Sub │ Dashboard Sub │ Query Sub │ AI Sub   │
│ /graphql │ /graphql      │ /graphql  │ /graphql │
└──────────┴──────────┴──────────┴──────────┘
      ↓ parallel fetch where possible
Router assembles response → returns to client
```

3.2 Core Schema Types

GraphQL Type	Key Fields & Description
Tenant	id, name, slug, tenantType (AGENCY ENTERPRISE), config: TenantConfig, subTenants: [SubTenant]
User	id, tenantId, email, role: Role, scope: Scope, invitedAt, lastActiveAt
Dashboard	id, tenantId, ownerId, title, spec: DashboardSpec, visibility: Visibility, schedule: Schedule
Widget	id, dashboardId, type: WidgetType, config: WidgetConfig, position: GridPosition
DashboardSpec	JSON — full dashboard layout spec: widgets[], gridLayout, dateRange, filters
PerformanceData	campaignId, date, channel, spend, impressions, clicks, conversions, roas, cpa, ctr
Connector	id, tenantId, type: ConnectorType, status: SyncStatus, lastSyncAt, accountIds
Recommendation	type: AlertType, metric, message, severity: Severity, value, baseline, delta
TenantConfig	logoUrl, brandColors, customDomain, ssoConfig, defaultCurrency, locale, whiteLabelMode

3.3 Key Query Operations

Operation	Signature & Purpose
dashboard	dashboard(id: ID!): Dashboard — Fetch single dashboard with full spec and widget configs
dashboards	dashboards(tenantId: ID!, filter: DashboardFilter): [Dashboard] — List all dashboards for tenant
widgetData	widgetData(widgetId: ID!, dateRange: DateRange!): WidgetData — Live Snowflake Gold layer data for one widget
dashboardData	dashboardData(dashboardId: ID!, dateRange: DateRange!): [WidgetData] — All widget data in one query (parallel)
channelSummary	channelSummary(tenantId: ID!, dateRange: DateRange!): ChannelSummary — Cross-channel Gold roll-up
recommendations	recommendations(tenantId: ID!): [Recommendation] — Current anomalies vs baseline
connectors	connectors(tenantId: ID!): [Connector] — List connectors + sync status
me	me: User — Current authenticated user + tenant context
tenantConfig	tenantConfig(tenantId: ID!): TenantConfig — Branding, SSO, locale config

3.4 Key Mutation Operations

Operation	Signature & Purpose
generateDashboard	generateDashboard(prompt: String!, tenantId: ID!): DashboardSpec — Claude AI prompt to full dashboard spec
saveDashboard	saveDashboard(spec: DashboardSpecInput!): Dashboard — Persist dashboard spec to storage
addWidget	addWidget(dashboardId: ID!, config: WidgetConfigInput!): Widget — Add single widget to dashboard
updateLayout	updateLayout(dashboardId: ID!, layout: [GridPositionInput!]!): Dashboard — Save drag/drop positions
shareDashboard	shareDashboard(dashboardId: ID!, input: ShareInput!): Dashboard — Set visibility, add recipients
scheduleDashboard	scheduleDashboard(dashboardId: ID!, schedule: ScheduleInput!): Dashboard — Configure PDF email cadence
createConnector	createConnector(input: ConnectorInput!): Connector — Auth + configure new data source
provisionTenant	provisionTenant(input: TenantInput!): Tenant — Trigger automated tenant setup pipeline
inviteUser	inviteUser(tenantId: ID!, email: String!, role: Role!): User — Send invitation + create user

updateTenantConfig

updateTenantConfig(tenantId: ID!, config: TenantConfigInput!): TenantConfig —
Update branding/SSO

3.5 Auth Context on Every Request

The Apollo Router validates the JWT on every incoming request and injects the auth context before forwarding to subgraphs. No subgraph performs its own auth — they trust the context set by the gateway.

```
// Context injected by Apollo Router into every subgraph request
interface AuthContext {
  userId:    string;           // Firebase Auth UID
  tenantId:  string;           // Tenant the user belongs to
  role:      Role;             // Admin | Manager | Analyst | Viewer | ClientUser
  scope: {
    type: 'all' | 'client' | 'brand' | 'region' | 'bu';
    ids: string[];             // Specific sub-tenant IDs this user can see
  };
  plan:      Plan;             // Starter | Growth | Scale | Enterprise
}
```

4. Data Architecture

MarketIQ uses a strict separation between analytical data (Snowflake) and operational/application data (Cloud SQL PostgreSQL). The two systems never share schemas and are accessed by different services.

4.1 Data Store Responsibilities

Data Store	Type	Stores
Snowflake	OLAP — Analytics Warehouse	All marketing performance data · Bronze/Silver/Gold layers · Per-tenant workspaces · Read-only from Query Service
Google Cloud Storage	Object Storage — Data Lake	Raw connector dumps (Bronze) · PDF exports · White-label assets · Logo uploads
Cloud SQL PostgreSQL	OLTP — Operational Database	Tenant configs · Users + roles · Dashboard specs · Connector credentials (encrypted) · Audit logs
Memorystore Redis	Cache + Queue Backend	Gold layer query result cache · BullMQ job queue backend · Session tokens · Rate limit counters
GCP Secret Manager	Secrets	Connector OAuth tokens · API keys · Service account credentials · Database passwords

4.2 Snowflake — Analytics Data Architecture

Multi-Tenant Workspace Structure

Snowflake Account

```

├─ DATABASE: marketiq_shared    ← Platform-wide reference data
|   └─ SCHEMA: dim_currency
|   └─ SCHEMA: dim_channel
|   └─ SCHEMA: dim_date
|
├─ DATABASE: tenant_{tenant_id} ← One database per tenant (complete isolation)
|   └─ SCHEMA: bronze           ← Raw data from Airbyte. Immutable. Append-only.
|   └─ SCHEMA: silver           ← Cleaned, normalised, deduplicated. dbt models.
|   └─ SCHEMA: gold             ← Aggregated KPIs, trend baselines. Dashboard queries.
|
└─ DATABASE: tenant_{tenant_id_2} ← Next tenant — fully isolated

```

Bronze Layer — Raw Ingestion

Immutable append-only tables. One table per connector per entity type. Populated by Airbyte. Never modified after write.

Table Pattern	Example	Contents
bronze.{connector}_{entity}	bronze.google_ads_campaign_performance	Raw JSON from API. All original fields preserved. ingested_at timestamp. source_account_id.
bronze.meta_ads_campaigns	bronze.meta_ads_ad_performance	All Meta fields verbatim. No field transformation. Replayable if Gold pipeline fails.
bronze.hubspot_contacts	bronze.hubspot_deals	CRM raw exports. Full field set including custom fields.

Silver Layer — Normalised

All channels mapped to the Universal Marketing Schema by dbt models. Deduplicated. Currency normalised. Type-cast.

```
-- silver.campaigns (Universal schema — identical structure regardless of source)
campaign_id          VARCHAR      -- MarketIQ internal ID (hash of source_id + channel +
tenant)
source_campaign_id   VARCHAR      -- Platform's original ID
tenant_id            VARCHAR      -- Tenant isolation key
channel              VARCHAR      -- google_ads | meta | linkedin | tiktok | ...
campaign_name        VARCHAR
objective            VARCHAR      -- awareness | traffic | leads | conversions | revenue
status              VARCHAR      -- active | paused | ended
start_date           DATE
end_date             DATE
daily_budget_usd     DECIMAL      -- Normalised to USD
lifetime_budget_usd  DECIMAL
currency_original    VARCHAR      -- Source currency code
created_at           TIMESTAMP
updated_at           TIMESTAMP
```

Gold Layer — Aggregated KPIs

Gold Table	Schema & Purpose
GOLD_CAMPAIGN_DAILY	date, tenant_id, campaign_id, channel, spend_usd, impressions, clicks, conversions, conversion_value_usd, reach, frequency, video_views, ctr, cpc_usd, cpm_usd, cpa_usd, roas, cvr — Pre-computed daily performance per campaign. Primary table for all dashboard queries.
GOLD_CHANNEL_SUMMARY	date, tenant_id, channel, total_spend_usd, total_impressions, total_clicks, total_conversions, blended_roas, blended_cpa, budget_daily_usd, budget_pacing_pct — Cross-channel daily roll-up. One row per channel per day per tenant.
GOLD_TREND_BASELINE	tenant_id, channel, metric, rolling_4w_avg, rolling_8w_avg, rolling_12w_avg, stddev, upper_threshold, lower_threshold, computed_at — Rolling averages and statistical thresholds. Powers all anomaly detection. Recomputed nightly.

GOLD_CAMPAIGN_PACING

date, tenant_id, campaign_id, budget_daily_usd, spend_to_date_usd, days_elapsed, days_remaining, projected_spend_usd, pacing_status — Budget pacing table. Flags over/under-pacing campaigns.

4.3 dbt Transformation Models

All Bronze-to-Silver and Silver-to-Gold transformations are version-controlled dbt models. Each connector has its own dbt source definition and staging model. Gold models are shared across all connectors.

```

data/dbt/
├── models/
│   ├── staging/                    ← Bronze → normalised staging (one per connector
│   │   │   entity)
│   │   ├── stg_google_ads_campaigns.sql
│   │   ├── stg_google_ads_performance.sql
│   │   ├── stg_meta_campaigns.sql
│   │   ├── stg_meta_performance.sql
│   │   └── ... (one per connector × entity)
│   ├── intermediate/              ← Cross-source joins, currency normalisation
│   │   ├── int_campaigns_unified.sql
│   │   └── int_performance_normalised.sql
│   └── marts/                      ← Gold layer tables
│       ├── gold_campaign_daily.sql
│       ├── gold_channel_summary.sql
│       ├── gold_trend_baseline.sql
│       └── gold_campaign_pacing.sql
├── sources/
│   └── bronze_sources.yml          ← Bronze table definitions per connector
├── tests/
│   └── data_quality.yml            ← Not null, unique, accepted values tests
└── dbt_project.yml

```

4.4 PostgreSQL — Operational Schema

Cloud SQL PostgreSQL stores all application state. Each microservice owns one or more schemas within the same PostgreSQL instance. Services access only their own schema via Prisma.

Service Schema	Tables
schema: tenants	tenants, sub_tenants, tenant_configs, tenant_plans, provisioning_jobs
schema: users	users, roles, scopes, invitations, sessions, user_preferences
schema: dashboards	dashboards, dashboard_versions, dashboard_shares, dashboard_schedules, templates
schema: widgets	widgets, widget_configs, widget_positions, widget_types
schema: connectors	connectors, connector_accounts, sync_jobs, sync_errors, connector_credentials (encrypted)

schema: audit

audit_events (append-only) — all user actions, config changes, data access events

4.5 Row-Level Security — Enforcement Pattern

Every Snowflake query executed by the Query Service automatically has tenant and scope filters appended. This is enforced in the Query Service middleware — not in individual resolver code.

```
// QueryService middleware — runs before every Snowflake execution
function injectTenantScope(sql: string, ctx: AuthContext): string {
  const tenantFilter = `tenant_id = '${ctx.tenantId}'`;

  const scopeFilter = ctx.scope.type === 'all'
    ? ''
    : `AND client_id IN (${ctx.scope.ids.map(id => `${id}`).join(',')})`;

  // Wrap original query in a scoped CTE
  return `
    WITH scoped AS (
      SELECT * FROM (${sql}) base
      WHERE ${tenantFilter} ${scopeFilter}
    )
    SELECT * FROM scoped
  `;
}

// Users cannot bypass this — SQL injection attempts are sanitised upstream
// Parameterised queries only — no string interpolation of user input
```

5. AI Service Architecture

The AI Service is the only service with access to the Claude API. It orchestrates all AI-powered functionality: prompt-to-dashboard spec generation, intent parsing, and natural language recommendation summaries.

5.1 Prompt-to-Dashboard Pipeline

```

User prompt (natural language)
  ↓
1. INTENT PARSER
  Claude call #1 – System: 'You are a marketing analytics assistant.'
  Output: structured IntentObject {
    metrics:    ['roas', 'spend', 'cpa'],
    dimensions: ['channel', 'campaign'],
    timeRange:  { type: 'relative', days: 90 },
    filters:    [],
    special:    ['highlight_underperforming'],
    implicit:   ['summary_kpis_at_top']
  }
  ↓
2. SPEC GENERATOR
  Claude call #2 – System: 'Generate a DashboardSpec JSON object.'
  Includes: widget types guide, available metrics list, grid schema
  Output: DashboardSpec { widgets: [...], layout: [...] }
  ↓
3. SPEC VALIDATOR
  Zod schema validation – reject malformed specs before query generation
  Fallback: retry with corrective prompt if validation fails (max 2 retries)
  ↓
4. QUERY BUILDER
  For each widget: WidgetConfig → parameterised Snowflake SQL
  tenant_id + scope injected by QueryService middleware
  ↓
5. PARALLEL EXECUTION
  All widget queries execute simultaneously via Promise.all()
  Streaming: spec chunks sent to client as SSE – progressive render
  ↓
6. RESPONSE
  DashboardSpec + WidgetData[] returned to frontend
  Frontend renders widgets as data arrives

```

5.2 Spec Caching Strategy

Identical prompts for the same tenant and date range should not regenerate specs — they should return the cached spec and refresh only the data.

Cache Layer	Strategy
-------------	----------

Spec cache (Redis)	Key: hash(tenantId + prompt + dateRange). TTL: 24 hours. On hit: return cached spec, execute queries with fresh data.
Query result cache (Redis)	Key: hash(tenantId + SQL + dateRange). TTL: 6 hours (matches default sync frequency). Invalidated on new sync completion.
No-cache scenarios	Date ranges ending 'today' bypass query cache. Force-refresh flag in GraphQL query bypasses both caches.

5.3 Claude API Configuration

```
// AI Service – Claude API config
const anthropic = new Anthropic({ apiKey: process.env.CLAUDE_API_KEY });

const INTENT_SYSTEM_PROMPT = `
  You are a marketing analytics assistant for MarketIQ.
  Extract structured intent from marketing dashboard requests.
  Always respond with valid JSON matching the IntentObject schema.
  Available metrics: ${METRICS_LIST}
  Available dimensions: ${DIMENSIONS_LIST}
  Available channels: ${CHANNEL_LIST}
`;

const SPEC_SYSTEM_PROMPT = `
  Generate a DashboardSpec JSON object.
  Available widget types: ${WIDGET_TYPES}
  Grid: 12 columns, rows unlimited.
  Always include at least 2 KPI cards at row 1.
  Respond with valid JSON only – no markdown, no preamble.
`;

// Intent parsing call
const intentResponse = await anthropic.messages.create({
  model: 'claude-sonnet-4-5',
  max_tokens: 800,
  system: INTENT_SYSTEM_PROMPT,
  messages: [{ role: 'user', content: userPrompt }]
});

// Spec generation call (streaming)
const specStream = await anthropic.messages.stream({
  model: 'claude-sonnet-4-5',
  max_tokens: 2000,
  system: SPEC_SYSTEM_PROMPT,
  messages: [{ role: 'user', content: JSON.stringify(intent) }]
});
```

6. Connector & Data Pipeline Architecture

The data pipeline ingests raw marketing data from external APIs, normalises it to the Universal Marketing Schema, and transforms it through Bronze, Silver, and Gold layers in Snowflake.

6.1 Pipeline Overview

```

External Marketing API (Google Ads, Meta, LinkedIn, etc.)
  ↓ OAuth 2.0 / API key (stored in Secret Manager)
Airbyte (managed extraction layer)
  · Handles: API pagination · Rate limiting · Auth token refresh · Retry on failure
  · Outputs: raw JSON to GCS Bronze bucket per connector per tenant
  ↓
GCS Bronze Bucket (tenant_{id}/bronze/{connector}/{entity}/{date}/)
  ↓ triggered by GCS object finalize event
Airflow DAG (Cloud Composer)
  · Loads GCS files into Snowflake bronze schema
  · Triggers dbt run for staging models
  ↓
dbt Staging Models (Bronze → Silver)
  · Field mapping to Universal Schema
  · Currency normalisation via DIM_CURRENCY
  · Deduplication on source_id + date + tenant_id
  ↓
dbt Mart Models (Silver → Gold)
  · Aggregation: GOLD_CAMPAIGN_DAILY, GOLD_CHANNEL_SUMMARY
  · GOLD_TREND_BASELINE updated nightly by Baseline Computer worker
  ↓
Gold Layer available for Query Service
  · Pub/Sub event: sync.completed published for each tenant
  · Notification Worker emails user if first sync or anomaly detected
  
```

6.2 Connector Standard Interface

```

// Every connector implements this interface
interface MarketIQConnector {
  type:      ConnectorType;           // e.g. 'google_ads' | 'meta'
  version:   string;

  // OAuth flow helpers
  getAuthUrl(tenantId: string): string;
}
  
```

```
exchangeCode(code: string, tenantId: string): Promise<OAuthTokens>;
refreshToken(tokens: OAuthTokens): Promise<OAuthTokens>;

// Airbyte config generation (we wrap Airbyte, not expose it)
toAirbyteConfig(credentials: ConnectorCredentials): AirbyteSourceConfig;

// Normalisation – maps raw Airbyte output to Universal Schema
normalise(raw: AirbyteSyncRecord[]): UniversalPerformanceRecord[];

// Validation
validateCredentials(creds: ConnectorCredentials): Promise<boolean>;
getSupportedDateRange(): { minDate: Date; maxDate: Date };
}
```

6.3 Sync Job State Machine

State	Description & Transitions
IDLE	No sync in progress. Waiting for scheduled trigger or manual trigger.
QUEUED	Sync trigger fired. Job queued in BullMQ. Waiting for Airbyte worker capacity.
EXTRACTING	Airbyte pulling data from external API. Duration: 2–30 minutes depending on volume.
LOADING	Data in GCS. Airflow DAG loading into Snowflake Bronze.
TRANSFORMING	dbt models running. Bronze → Silver → Gold.
COMPLETED	Gold layer updated. sync.completed event published. Query cache invalidated.
FAILED	Error at any stage. Error logged. Notification sent. Retry queued with exponential backoff.
PAUSED	Connector manually paused by tenant admin. No syncs triggered until resumed.

7. Frontend Architecture

The frontend is a React + TypeScript single-page application built with Vite, served via Cloud CDN. It uses Apollo Client for GraphQL, Zustand for global state, and TanStack Query for server state management.

7.1 Application Structure

```
apps/web/src/
├── app/
│   ├── router.tsx          ← React Router v6 routes
│   ├── providers.tsx       ← Apollo, Auth, Theme, i18n providers
│   └── App.tsx
├── pages/
│   ├── auth/              ← Login, SSO callback, invite accept
│   ├── onboarding/        ← 5-step onboarding wizard
│   ├── dashboard/
│   │   ├── DashboardCanvas.tsx ← react-grid-layout canvas
│   │   ├── PromptBar.tsx     ← AI prompt input
│   │   ├── WidgetPanel.tsx   ← Right sidebar widget builder
│   │   └── DashboardList.tsx
│   ├── connectors/        ← Connector setup + OAuth flows
│   ├── settings/          ← Tenant config, white-label, SSO
│   └── admin/             ← RBAC management, billing
├── components/
│   ├── widgets/           ← All 10 widget type components
│   │   ├── KpiCard.tsx
│   │   ├── BarChart.tsx    ← Recharts
│   │   ├── LineChart.tsx   ← Recharts
│   │   ├── Heatmap.tsx     ← Apache ECharts
│   │   ├── FunnelChart.tsx ← Apache ECharts
│   │   └── ...
│   ├── anomaly/           ← Recommendation overlay cards
│   ├── layout/            ← Sidebar, topbar, nav
│   └── ui/                ← shadcn/ui overrides + brand theming
├── store/
│   ├── tenant.store.ts     ← Zustand: current tenant, branding
│   ├── user.store.ts       ← Zustand: current user, role, scope
│   └── ui.store.ts         ← Zustand: sidebar, modal, theme
├── graphql/
│   ├── queries/            ← All GQL query documents
│   ├── mutations/          ← All GQL mutation documents
│   └── generated/          ← Auto-generated TypeScript types (codegen)
├── lib/
│   ├── apollo.ts           ← Apollo Client config + auth link
│   ├── theme.ts            ← CSS variable injection from tenant config
│   └── i18n.ts             ← react-i18next config
└── styles/
    ├── globals.css         ← CSS custom properties (white-label vars)
    └── theme.css           ← shadcn/ui theme overrides
```

7.2 State Management Layers

Layer	Tool	What It Manages
Global client state	Zustand	Current tenant, user, role, scope, sidebar open/closed, active theme, locale — shared across all components
Server state	TanStack Query (React Query)	All data from GraphQL — dashboard specs, widget data, connector status. Handles caching, background refresh, loading/error states.
GraphQL queries	Apollo Client	GraphQL operations, normalized entity cache, optimistic UI for mutations. Works alongside TanStack Query.
Local component state	React useState/useReducer	Form state, modal open/close, widget config panel step — transient UI state not shared
URL state	React Router searchParams	Active dashboard ID, selected date range, active filters — shareable via URL

7.3 White-Label Theme System

The entire UI is themed via CSS custom properties injected at runtime from the tenant config object. shadcn/ui components reference these variables — no hard-coded brand colours anywhere in component code.

```
// lib/theme.ts — called at app boot after tenant config loads
export function applyTenantTheme(config: TenantConfig): void {
  const root = document.documentElement;
  root.style.setProperty('--color-primary',    config.brandColors.primary);
  root.style.setProperty('--color-secondary',  config.brandColors.secondary);
  root.style.setProperty('--color-accent',     config.brandColors.accent);
  root.style.setProperty('--color-bg',        config.brandColors.background ??
  '#F8FAFC');

  // Logo injection
  const logoEl = document.getElementById('tenant-logo');
  if (logoEl) logoEl.setAttribute('src', config.logoUrl);

  // Favicon
  const favicon = document.querySelector('link[rel=icon]') as HTMLLinkElement;
  if (favicon) favicon.href = config.faviconUrl ?? config.logoUrl;
}

// All shadcn/ui components use these variables:
// .bg-primary → var(--color-primary)
// Charts inherit --color-primary for default series colour
// No hex values appear in component JSX
```

8. Security Architecture

8.1 Authentication Flow

1. User visits `https://[tenant].marketiq.app`
2. App loads tenant config (logo, colours) from CDN-cached tenant config endpoint
3. User clicks 'Sign in'
- 4a. Standard: Firebase Auth email/password or Google OAuth
- 4b. SSO: Firebase Auth SAML provider redirect (Azure AD, Okta, Google Workspace)
5. Firebase issues short-lived ID token (1 hour) + long-lived refresh token
6. Frontend stores ID token in memory (NOT localStorage — XSS protection)
7. Every GraphQL request: Authorization: Bearer {idToken} header
8. Apollo Router validates token with Firebase Admin SDK
9. Router extracts: uid, tenantId (custom claim), role, scope (custom claims)
10. Auth context injected into all downstream subgraph requests
11. Token auto-refreshed 5 minutes before expiry via Firebase SDK

8.2 Token Architecture

Token Type	Storage · Lifetime · Usage
Firebase ID Token	Memory only (JS variable). 1 hour lifetime. Sent in every GraphQL request header.
Firebase Refresh Token	HttpOnly secure cookie. No JS access. 30-day lifetime. Used only to refresh ID token.
Connector OAuth Tokens	GCP Secret Manager only. Never in PostgreSQL or code. Accessed only by Connector Service via Secret Manager API.
Service Account Keys	GCP Secret Manager. Used by Cloud Run services for inter-GCP service auth. Rotated every 90 days.

8.3 Network Security

Layer	Controls
Perimeter	Google Cloud Armor — WAF rules, OWASP top 10 protection, IP allowlisting available for enterprise tenants
DDoS	Cloud Armor adaptive protection — automatic DDoS mitigation at L3/L4/L7
Transport	TLS 1.3 enforced on all endpoints. HSTS with 1-year max-age + includeSubDomains. No TLS 1.0/1.1.

API Rate Limiting	Per-tenant per-endpoint rate limits in Redis. GraphQL depth limiting (max depth 8). Query complexity limits.
Service Mesh	Cloud Run services on VPC — not publicly accessible. Only Apollo Router is public-facing.
Secrets	All credentials in Secret Manager. Environment variables in Cloud Run use Secret Manager references — never plaintext.

8.4 Multi-Tenant Isolation Layers

Tenant isolation is enforced at three independent layers — all three must be breached for a cross-tenant data leak:

1. JWT validation — tenant_id claim verified cryptographically by Firebase on every request
2. Application layer — AuthContext checked by every resolver before any data access
3. Database layer — Row-level security injected by Query Service middleware on every Snowflake query

8.5 Connector Security

Principle: MarketIQ operates with read-only OAuth scopes only.

We never write to, modify, or delete customer ad account data.

OAuth token lifecycle:

1. User authenticates OAuth flow in-app
2. Authorization code exchanged server-side (never exposed to browser)
3. Access token + refresh token stored ONLY in GCP Secret Manager
Key format: projects/{project}/secrets/connector-{tenantId}-{connectorId}
4. Connector Service retrieves token from Secret Manager at sync time
5. Token refresh handled server-side before expiry
6. Token revocation: deletes Secret Manager entry + notifies user

8.6 Audit Logging

All significant actions are logged to the audit schema in PostgreSQL and replicated to Cloud Logging for 90-day retention and SIEM integration.

```
// audit.audit_events table — append-only, never updated or deleted
CREATE TABLE audit.audit_events (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   VARCHAR NOT NULL,
  user_id     VARCHAR NOT NULL,
  action      VARCHAR NOT NULL, -- e.g. 'dashboard.view' | 'connector.create'
  resource_type VARCHAR,        -- e.g. 'dashboard' | 'connector' | 'user'
```

```
resource_id VARCHAR,  
ip_address  INET,  
user_agent  TEXT,  
request_id  VARCHAR,          -- Trace ID for correlation  
metadata    JSONB,            -- Action-specific detail  
created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX ON audit.audit_events (tenant_id, created_at DESC);  
CREATE INDEX ON audit.audit_events (user_id, created_at DESC);
```

9. Infrastructure Architecture — GCP

9.1 GCP Project Structure

GCP Project	Environment	Contains
marketiq-dev	Development	All dev services. Snowflake dev workspace. Feature branch deployments.
marketiq-staging	Staging	Production mirror. Used for integration testing + pre-launch validation. Separate Snowflake staging account.
marketiq-prod	Production	Live customer traffic. All production GCP resources. Production Snowflake account.
marketiq-shared	Shared	Artifact Registry (Docker images). Shared VPC. Terraform state bucket. Monitoring dashboards.

9.2 GCP Resource Inventory

GCP Service	Configuration	Purpose
Cloud Run	10 services · Min 0 replicas (gateway: 1) · Max 10 per service · 1 vCPU / 512MB default	Hosts all microservices. Scale to zero reduces costs significantly.
Cloud Functions (2nd gen)	5 functions · 1 vCPU / 512MB · Max 100 concurrent	Event-driven workers. Sync trigger, export, notification, provisioning, baseline.
Cloud SQL PostgreSQL	db-n1-standard-2 (prod) · HA with failover replica · Automated daily backups	Operational database for all application state.
Memorystore Redis	Standard tier · 5GB (prod) · 1 replica · 99.9% SLA	Query cache, BullMQ queue backend, rate limiting, sessions.
Cloud Pub/Sub	Topics per event type · Dead-letter topics · 7-day retention	Async messaging between services and workers.
Cloud Storage	Standard class · Multi-region (us/eu) · Object lifecycle policies	Data lake (Bronze), PDF exports, white-label assets.
Cloud CDN	Global · HTTPS · Custom origins per tenant subdomain	Static assets, SPA delivery, tenant subdomain routing.
Cloud Load Balancing	Global HTTPS LB · SSL cert auto-renewal · Wildcard cert for *.marketiq.app	Ingress, SSL termination, routing to Cloud Run services.
Cloud Armor	Security policy · OWASP rules · Adaptive DDoS protection	WAF and DDoS mitigation on all public endpoints.

Firestore Auth	Multi-tenant mode · SAML providers per tenant · Custom claims	Authentication, SSO per enterprise tenant, token issuance.
Secret Manager	CMEK encryption · Rotation policies · IAM access control	All credentials, API keys, OAuth tokens.
Artifact Registry	Docker repository per service · Vulnerability scanning	Container image storage and security scanning.
Cloud Composer (Airflow)	Composer 2 · Auto-scaling workers · Managed upgrades	dbt + Airbyte pipeline orchestration.
Cloud Monitoring	Custom dashboards · Alerting policies · Uptime checks	Infrastructure and application performance monitoring.
Cloud Logging	30-day retention (prod) · Log sinks to BigQuery for audit	Centralised logging across all services.

9.3 Network Architecture

```

Internet
  ↓ HTTPS only (port 443)
Cloud Load Balancer + Cloud Armor (WAF)
  ↓ SSL termination · Backend service routing

VPC: marketiq-vpc (10.0.0.0/8)

Subnet: public (10.0.1.0/24)
  └─ Cloud Run services (serverless VPC connector)
    └─ Apollo Router ← only public-facing service

Subnet: private (10.0.2.0/24)
  └─ Cloud SQL PostgreSQL (private IP only)
  └─ Memorystore Redis (private IP only)
  └─ Cloud Composer (Airflow)

Subnet: data (10.0.3.0/24)
  └─ Cloud Composer workers · dbt runners

Private Google Access: enabled (all subnets)
Cloud NAT: egress for Cloud Functions + Composer

↓ Snowflake Private Link (GCP PrivateLink)
Snowflake Cloud Account (GCP us-central1)

```

9.4 Multi-Region & Availability

Component	Availability Strategy
Cloud Run	Multi-zone by default. Cloud Run manages zone distribution automatically.

Cloud SQL	High availability config — primary + standby in separate zones. Automatic failover < 60 seconds.
Memorystore Redis	Standard tier with replica in separate zone. Automatic failover.
Cloud Storage	Multi-region bucket (us or eu depending on customer data residency requirement).
Cloud Pub/Sub	Globally replicated by default. Messages durable across zone failures.
Snowflake	Snowflake multi-cluster warehouse. Auto-suspend/resume. Business Critical edition for enterprise (SOC 2 + HIPAA).
DNS + CDN	Cloud DNS with global Anycast. Cloud CDN global PoPs. < 50ms TTFB from major markets.

10. Monorepo & Repository Structure

The entire platform lives in a single GitHub repository managed by Turborepo. Turborepo handles incremental builds, shared package caching, and parallel task execution across all apps, services, and workers.

10.1 Repository Layout

```
marketiq/ (root - Turborepo monorepo)
├── apps/
│   ├── web/                ← React + Vite frontend SPA
│   ├── gateway/           ← Apollo Federation Router config
│   └── services/           ← 10 Node.js microservices
│       ├── auth/
│       ├── tenant/
│       ├── user/
│       ├── dashboard/
│       ├── widget/
│       ├── query/
│       ├── ai/
│       ├── connector/
│       └── recommendation/
├── workers/                ← 5 Cloud Functions
│   ├── sync-trigger/
│   ├── export/
│   ├── notification/
│   ├── provisioning/
│   └── baseline/
├── packages/
│   ├── db/                ← Shared internal libraries
│   ├── snowflake/         ← Shared Prisma base config + migrations
│   ├── types/             ← Snowflake client + typed query builders
│   ├── ui/               ← Shared TypeScript interfaces
│   ├── shadcn/ui          ← shadcn/ui components (white-label aware)
│   ├── config/            ← Shared ESLint + TypeScript configs
│   └── testing/           ← Shared test utilities + fixtures
├── data/
│   ├── dbt/               ← Data pipeline definitions
│   ├── airflow/           ← dbt project (Bronze → Silver → Gold)
│   ├── airflow/           ← Airflow DAG definitions
│   └── airbyte/           ← Connector source configs
├── infra/
│   └── terraform/         ← All GCP infrastructure as code
│       ├── modules/
│       │   ├── cloud-run/
│       │   ├── cloud-sql/
│       │   ├── snowflake-workspace/ ← Per-tenant provisioning module
│       │   ├── pubsub/
│       │   └── networking/
```

```

├── environments/
│   ├── dev/
│   ├── staging/
│   └── prod/
└── tenant-provision/      ← Tenant provisioning Terraform workspace

├── .github/
│   └── workflows/        ← GitHub Actions CI/CD
│       ├── ci.yml        ← PR checks: lint, typecheck, test
│       ├── build.yml     ← Build Docker images on merge to main
│       └── deploy.yml    ← Deploy to staging + prod
├── turbo.json            ← Turborepo pipeline config
├── pnpm-workspace.yaml   ← pnpm workspace config
└── package.json          ← Root scripts

```

10.2 Turborepo Pipeline

```

// turbo.json - task dependency graph
{
  'pipeline': {
    'build': {
      'dependsOn': ['^build'],      // Build dependencies first
      'outputs': ['dist/**', '.next/**']
    },
    'typecheck': {
      'dependsOn': ['^build']      // Types must be built before checking
    },
    'test': {
      'dependsOn': ['build'],
      'outputs': ['coverage/**']
    },
    'lint': {
      'outputs': []
    },
    'docker:build': {
      'dependsOn': ['build'],
      'outputs': []
    }
  }
}

// On PR: turbo run lint typecheck test --filter=[HEAD^1]
// Only runs tasks for changed packages and their dependents

```

11. CI/CD Pipeline

11.1 Branch Strategy

Branch	Purpose & Rules
main	Production-ready code. Protected — requires PR + 1 review + all CI checks passing. Auto-deploys to staging on merge.
release/*	Release branches. Tagged for production deployment. Requires explicit prod deploy approval.
feature/*	Feature branches. PR target: main. Turbo cache speeds up CI — only changed packages tested.
hotfix/*	Emergency fixes. Fast-track PR process. Direct deploy to prod after review.

11.2 CI Pipeline — GitHub Actions

```
# .github/workflows/ci.yml
on: pull_request

jobs:
  lint-and-typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3
      - run: pnpm install --frozen-lockfile
      - run: pnpm turbo run lint typecheck --filter=[HEAD^1]

  test:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:16, env: { POSTGRES_PASSWORD: test } }
      redis:    { image: redis:7 }
    steps:
      - run: pnpm turbo run test --filter=[HEAD^1]
      - uses: codecov/codecov-action@v4

  build-docker:
    runs-on: ubuntu-latest
    needs: [lint-and-typecheck, test]
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    steps:
      - uses: google-github-actions/auth@v2
        with: { credentials_json: '${{ secrets.GCP_SA_KEY }}' }
      - run: |
          for service in auth tenant user dashboard widget query ai connector
          recommendation; do
```

```
docker build -t gcr.io/marketiq-prod/$service:${{ github.sha }}
apps/services/$service
docker push gcr.io/marketiq-prod/$service:${{ github.sha }}
done
```

11.3 Deployment Pipeline

Stage	Trigger	Process
Deploy to Staging	Merge to main (automatic)	Docker images pushed → Terraform plan → Cloud Run services updated → Integration tests → Smoke tests → Slack notification
Deploy to Production	Manual approval in GitHub Actions	Requires explicit approval from 2 engineers · Terraform apply · Rolling update per service · Health check verification · Rollback on failure
Tenant Provisioning	API call from Tenant Service	Terraform workspace apply for new tenant · Snowflake workspace creation · DNS record + SSL cert · Airbyte source creation
dbt Deployment	PR to data/dbt/**	dbt Cloud CI job runs → tests on staging → manual approve → production dbt run

12. Monitoring & Observability

12.1 Observability Stack

Tool	What It Monitors
Cloud Monitoring	Infrastructure metrics: Cloud Run CPU/memory/request count, Cloud SQL query latency, Redis hit rate, Pub/Sub message lag, Cloud Functions execution time/errors
Cloud Logging	Structured JSON logs from all services. Centralised. Queryable. 30-day retention (prod). Audit logs shipped to Cloud Storage for 1-year retention.
Sentry	Frontend: unhandled exceptions, React error boundaries, Core Web Vitals. Backend: uncaught exceptions with full stack traces and request context.
Cloud Trace	Distributed tracing across GraphQL requests through multiple services. Identifies slow resolvers and N+1 query patterns.
Snowflake Query History	Slow query detection. Per-tenant compute usage. Credits consumption. Used for cost management and query optimisation.
Uptime Checks	External probes to /health endpoint on Apollo Router every 60 seconds from 3 regions. PagerDuty alert if 2+ consecutive failures.

12.2 Key SLOs & Alerting Thresholds

Metric	SLO Target	Alert Threshold	Alert Channel
API p95 latency	< 500ms	> 800ms for 5+ minutes	PagerDuty + Slack
Gold query p95 latency	< 2 seconds	< 4 seconds for 5+ minutes	PagerDuty + Slack
API error rate	< 0.5%	> 1% for 2+ minutes	PagerDuty
Connector sync success	> 99%	< 95% over 1 hour	Slack + email to tenant
Cloud Run availability	> 99.9%	Any 5xx > 5% for 2+ minutes	PagerDuty
Pub/Sub message lag	< 30 seconds	< 5 minutes unprocessed	Slack
Redis memory usage	< 80%	> 90%	Slack
Snowflake credit burn	Within budget	Unexpected 2x hourly spike	Slack

12.3 Health Check Endpoints

Every Cloud Run service exposes a /health endpoint checked by the load balancer and uptime monitoring:

```
// Standardised health check – every service
app.get('/health', async (req, res) => {
  const checks = await Promise.allSettled([
    db.$queryRaw`SELECT 1`,           // PostgreSQL connectivity
    redis.ping(),                     // Redis connectivity
    // Service-specific checks:
    // Query Service: Snowflake test query
    // AI Service: Claude API reachability
    // Connector Service: Secret Manager access
  ]);

  const allHealthy = checks.every(c => c.status === 'fulfilled');

  res.status(allHealthy ? 200 : 503).json({
    status:    allHealthy ? 'healthy' : 'degraded',
    service:   process.env.SERVICE_NAME,
    version:   process.env.GIT_SHA,
    timestamp: new Date().toISOString(),
    checks:    checks.map((c, i) => ({
      name:    ['postgres', 'redis', ...serviceSpecific][i],
      status:  c.status === 'fulfilled' ? 'pass' : 'fail',
    }))
  });
});
```

13. Technology Stack Reference

13.1 Frontend

Package	Version Strategy	Purpose	Notes
react	^18.x	UI framework	Server components not used — pure SPA
typescript	^5.x	Type safety	Strict mode enabled across all apps
vite	^5.x	Build + dev server	ESBuild for transforms, Rollup for prod bundle
@apollo/client	^3.x	GraphQL client	With normalised cache + reactive variables
shadcn/ui	latest	Component library	Unstyled — white-label via CSS variables
recharts	^2.x	Charts (standard)	KPI, Bar, Line, Pie, Area
echarts-for-react	^3.x	Charts (complex)	Heatmap, Funnel, Scatter, Radar
react-grid-layout	^1.x	Dashboard canvas	Drag/resize/persist layout as JSON
zustand	^4.x	Global state	Lightweight, no Redux boilerplate
@tanstack/react-query	^5.x	Server state	Background refresh, stale-while-revalidate
react-i18next	^14.x	i18n	Lazy locale loading, namespace support
@graphql-codegen/cli	^5.x	Type generation	Auto TypeScript types from GQL schema
@azure/msal-react	^2.x	SSO	Enterprise Azure AD SSO support

13.2 Backend (per service)

Package	Version	Purpose	Notes
node	20 LTS	Runtime	All services + workers
typescript	^5.x	Type safety	Shared tsconfig from packages/config
@apollo/server	^4.x	GraphQL server	One instance per microservice

@apollo/subgraph	^2.x	Federation	Subgraph directive support
@pothos/core	^3.x	Schema builder	Code-first, TypeScript-native
prisma	^5.x	ORM	Per-service schema, auto migrations
@prisma/client	^5.x	DB client	Generated from per-service schema.prisma
zod	^3.x	Validation	Input validation + type inference
bullmq	^5.x	Job queues	For services needing async internal jobs
ioredis	^5.x	Redis client	Used by Query Service + BullMQ
@anthropic-ai/sdk	^0.x	Claude API	AI Service only
@google-cloud/pubsub	^4.x	Pub/Sub	All services for event publishing
@google-cloud/secret-manager	^5.x	Secrets	Connector Service for OAuth tokens
snowflake-sdk	^1.x	Snowflake	Query Service only

13.3 Data Stack

Technology	Version / Config	Role
Snowflake	Standard edition · GCP us-central1	Analytics warehouse — all Bronze/Silver/Gold layers
dbt	dbt Core 1.8.x	SQL transformations — Bronze to Silver to Gold
Apache Airflow	Cloud Composer 2 (managed)	Pipeline orchestration — schedules dbt runs + Airbyte syncs
Airbyte	Cloud / Self-hosted 0.x	Connector extraction — wraps all external marketing APIs
Cloud SQL	PostgreSQL 16 · db-n1-standard-2	Operational DB — application state and audit logs
Memorystore Redis	7.x · Standard 5GB	Query cache · BullMQ backend · Rate limiting
Google Cloud Storage	Standard class · Multi-region	Data lake (Bronze raw files) · PDF exports · Assets

13.4 DevOps

Tool	Version	Purpose
Turborepo	^2.x	Monorepo build system — incremental builds, remote caching
pnpm	^9.x	Package manager — workspace support, fast installs

Docker	24.x	Container runtime — all services containerised
Terraform	^1.8.x	GCP infrastructure as code — all resources, tenant provisioning
GitHub Actions	Latest	CI/CD — lint, test, build, deploy workflows
Puppeteer	^22.x	PDF export — headless Chrome in Cloud Functions
Resend	^3.x	Transactional email — dashboard reports, notifications, onboarding
Sentry	^8.x	Error tracking — frontend + backend with release tracking

14. Environment Configuration

All environment variables are injected at runtime via GCP Secret Manager references in Cloud Run. No secrets are stored in source code or .env files in version control.

14.1 Common Variables (all services)

```
NODE_ENV=production
SERVICE_NAME=dashboard-service           # Injected by Terraform
GIT_SHA=${GIT_SHA}                        # Injected by CI/CD at deploy time
GOOGLE_CLOUD_PROJECT=marketiq-prod

# Injected from Secret Manager
DATABASE_URL=postgresql://...            # Cloud SQL connection string
REDIS_URL=redis://...                    # Memystore private IP
PUBSUB_PROJECT_ID=marketiq-prod
```

14.2 Service-Specific Variables

Service	Additional Variables
AI Service	ANTHROPIC_API_KEY (Secret Manager ref) · CLAUDE_MODEL=claude-sonnet-4-5 · MAX_SPEC_RETRIES=2
Query Service	SNOWFLAKE_ACCOUNT · SNOWFLAKE_USER · SNOWFLAKE_PRIVATE_KEY (Secret Manager ref) · SNOWFLAKE_WAREHOUSE · CACHE_TTL_SECONDS=21600
Connector Service	AIRBYTE_API_URL · AIRBYTE_API_KEY (Secret Manager ref) · SECRET_MANAGER_PREFIX=projects/marketiq-prod/secrets
Auth Service	FIREBASE_PROJECT_ID · FIREBASE_SERVICE_ACCOUNT (Secret Manager ref) · JWT_AUDIENCE
Notification Worker	RESEND_API_KEY (Secret Manager ref) · FROM_EMAIL_DEFAULT=noreply@marketiq.app · PUPPETEER_EXECUTABLE_PATH
Tenant Service	TERRAFORM_CLOUD_TOKEN (Secret Manager ref) · TERRAFORM_WORKSPACE_PREFIX=tenant-provision

15. Architecture Decision Records (ADRs)

Key architectural decisions with context and rationale. ADRs are numbered and immutable — superseded decisions are marked as such.

ADR-001 — GCP over Azure

Attribute	Detail
Decision	Use Google Cloud Platform as primary cloud provider
Status	Accepted
Context	Initial design used Azure and Microsoft Fabric. Customer profile is white-label SaaS — cloud brand is invisible to end users. Cost and capability matter; brand alignment does not.
Reasoning	GCP is 35–40% cheaper at scale. BigQuery/Snowflake on GCP has better price/performance than Fabric. Cloud Run is simpler than Azure Container Apps. Firebase Auth is better for multi-tenant than Azure AD B2C.
Consequences	Fabric/Purview governance story lost. Rebuilt as Snowflake + dbt + Terraform governance. 1-week additional setup vs Azure.

ADR-002 — GraphQL Federation over REST or tRPC

Attribute	Detail
Decision	Use Apollo Federation v2 with GraphQL as the API layer
Status	Accepted
Context	Dashboard with 8-12 widgets needs data from multiple services simultaneously. REST would require multiple sequential calls or a custom BFF.
Reasoning	GraphQL lets clients request exactly the data they need. Federation allows each service to own its schema without a monolithic API. Dashboard queries can fetch all widget data in a single network round-trip. Type generation via codegen eliminates frontend-backend type drift.
Consequences	Apollo Federation Router adds operational complexity. Team must understand Federation concepts (entity references, subgraphs). Cold-start latency on first request after scale-to-zero.

ADR-003 — Snowflake over BigQuery for Analytics

Attribute	Detail
Decision	Use Snowflake (hosted on GCP) as the analytics warehouse
Status	Accepted

Context	BigQuery is cheaper for consumption-based pricing. Snowflake adds cost but has different capabilities.
Reasoning	Snowflake's workspace-per-database model maps perfectly to workspace-per-tenant isolation requirement. Snowflake's RBAC is more granular than BigQuery's. Row-level security is more mature. Better multi-cluster warehousing for per-tenant query isolation.
Consequences	~20–30% higher analytics cost vs BigQuery at equivalent scale. Justified by cleaner tenant isolation architecture and reduced security engineering effort.

ADR-004 — Microservices over Monolith

Attribute	Detail
Decision	Decompose into 10 independent microservices from day one
Status	Accepted
Context	Could start as a monolith and extract services later. Claude Code accelerates boilerplate generation, reducing the cost of microservice setup.
Reasoning	Services can scale independently — Query Service needs more memory than Auth Service. Independent deployments reduce risk per deploy. Schema ownership per service enforces correct data isolation. Claude Code generates scaffold for all services in minutes, eliminating the traditional monolith-first argument.
Consequences	More operational complexity. More repos/configs to manage (mitigated by Turborepo). Service-to-service communication rules must be enforced.

ADR-005 — Claude API for AI Dashboard Generation

Attribute	Detail
Decision	Use Anthropic Claude API (claude-sonnet-4-5) for all AI-powered features
Status	Accepted
Context	Could use OpenAI GPT-4o or Google Gemini (which would be native to GCP).
Reasoning	Claude produces more consistently structured JSON output for spec generation. Claude's context window handles complex dashboard prompts with full schema context. Claude is cloud-agnostic — not locked to GCP or Azure. Anthropic's safety and reliability track record is strong.
Consequences	External API dependency — not native to GCP. Network egress cost. Must handle API downtime gracefully (cache fallback + user-friendly error).

End of Document

MarketIQ · Technical Architecture Document · v1.0 · April 2025